

Redis Learning Guide

1. What is Redis?

Redis (Remote Dictionary Server) is an **in-memory data structure store** used as:

- Database
- Cache
- Message broker
- Real-time data store

Unlike traditional databases that store data on disk, Redis stores data **in RAM**, making it **extremely fast (microseconds latency)**.

Typical use cases:

- API caching
 - session storage
 - pub/sub messaging
 - leaderboards
 - rate limiting
 - job queues
-

2. Why Redis is Fast

Redis is fast because:

1. Data is stored **in memory (RAM)**
2. Uses **single-threaded event loop**
3. Uses **efficient data structures**
4. Minimal disk access

Typical latency:

- ~1 millisecond or less
-

3. Installing Redis

Ubuntu

```
sudo apt update
sudo apt install redis-server
```

Start Redis

```
redis-server
```

Test Redis

```
redis-cli ping
```

Output:

```
PONG
```

4. Redis Architecture

Redis works as a **client-server model**.

```
Application
  |
  v
Redis Client
  |
  v
Redis Server (In Memory)
```

Multiple applications can connect to the same Redis server.

5. Redis Data Types

Redis is not just key-value.

It supports multiple data structures.

Type	Description
String	Basic key value
List	Ordered collection
Set	Unique unordered values
Sorted Set	Ordered by score
Hash	Object-like structure
Stream	Event stream
Bitmap	Bit-level storage
HyperLogLog	Approximate counting

6. Redis Keys

All Redis data is stored as:

```
key -> value
```

Example

```
user:1:name -> "Arpit"
```

Keys are typically **namespaced**.

Example:

```
user:1001  
session:token  
cart:45
```

7. Redis String Commands

Strings are the **most basic Redis data type**.

Set value

```
SET name "Arpit"
```

Get value

```
GET name
```

Delete key

```
DEL name
```

Check if key exists

```
EXISTS name
```

Increment

```
INCR counter
```

Decrement

```
DECR counter
```

Example

```
SET visits 1  
INCR visits
```

8. Expiring Keys (TTL)

Redis allows automatic expiration.

Example:

```
SET session:123 "user_data"  
EXPIRE session:123 60
```

This key will expire in **60 seconds**.

Check TTL

```
TTL session:123
```

9. Redis Lists

Lists are ordered collections.

Example use cases:

- message queues
- activity logs

Add to left

```
LPUSH tasks "task1"
```

Add to right

```
RPUSH tasks "task2"
```

View list

```
LRANGE tasks 0 -1
```

Pop element

```
LPOP tasks
```

10. Redis Sets

Sets store **unique values**.

Example:

```
SADD users "arpit"
```

```
SADD users "rahul"
```

View members:

```
SMEMBERS users
```

Check membership:

```
SISMEMBER users "arpit"
```

11. Redis Hashes

Hashes store **objects**.

Example:

```
HSET user:1 name "Arpit"  
HSET user:1 age 22
```

Get all fields:

```
HGETALL user:1
```

Get specific field:

```
HGET user:1 name
```

12. Redis Sorted Sets

Sorted sets store data with a **score**.

Example leaderboard.

```
ZADD leaderboard 100 arpit  
ZADD leaderboard 80 rahul
```

View leaderboard:

```
ZRANGE leaderboard 0 -1 WITHSCORES
```

Top score:

```
ZREVRANGE leaderboard 0 2 WITHSCORES
```

13. Redis Pub/Sub

Redis supports messaging.

Publisher:

```
PUBLISH chat "hello"
```

Subscriber:

```
SUBSCRIBE chat
```

Used for:

- live notifications
 - chat systems
 - event systems
-

14. Redis Persistence

Although Redis is in-memory, it supports persistence.

RDB (Snapshot)

Saves snapshots periodically.

AOF (Append Only File)

Logs every write operation.

15. Redis Use Cases

Caching

Example:

```
GET user_profile
```

If not found:

```
Fetch from database  
Store in Redis
```

Rate Limiting

```
INCR api:requests:user1  
EXPIRE api:requests:user1 60
```

Session Store

```
SET session:abc user123 EX 3600
```

Queue System

```
LPUSH queue job1  
BRPOP queue
```

16. Redis with Python

Install library

```
pip install redis
```

Example:

```
import redis  
  
r = redis.Redis(host='localhost', port=6379)  
  
r.set("name", "Arpit")  
print(r.get("name"))
```

17. Redis with Node.js

Install

```
npm install redis
```

Example:

```
import { createClient } from "redis";  
  
const client = createClient();  
  
await client.connect();  
  
await client.set("name", "Arpit");  
  
const value = await client.get("name");  
  
console.log(value);
```

18. Redis Interview Level Concepts

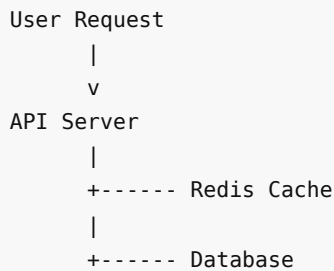
Important concepts:

- Redis eviction policies
- Redis clustering
- Redis replication
- Redis transactions

- Redis pipelines
- Redis Lua scripting
- Redis streams

19. Real World System Example

Example architecture:



Flow:

1. Check Redis
2. If cache miss -> fetch DB
3. Store in Redis
4. Return response

20. What to Learn Next

After basics:

1. Redis caching patterns
2. Redis streams
3. Redis cluster
4. Redis pub/sub systems
5. Redis rate limiting
6. Redis distributed locks
7. Redis for real-time analytics

End of Redis Basics

Redis Deep Dive — Part 1

Redis Internal Architecture

Understanding Redis architecture is the **key difference between a Redis user and a Redis engineer.**

1. High Level Redis Architecture

Redis follows a **single-process event-driven architecture**.

```
Client
|
v
TCP Connection
|
v
Redis Event Loop
|
+---- Command Parser
|
+---- In-Memory Data Store
|
+---- Persistence Engine
```

Redis handles **thousands of clients concurrently using a single thread**.

2. Why Redis Uses Single Thread

Most people assume:

Single thread = slow

But Redis proves the opposite.

Reasons:

1. No Locking

Multi-threaded systems require:

- mutex
- locks
- context switching

Redis avoids all of that.

```
Thread A -> lock key
Thread B -> wait
Thread C -> wait
```

Redis avoids this entire problem.

2. CPU Cache Friendly

A single thread means:

- better CPU cache locality

- fewer context switches

This dramatically improves speed.

3. Network Bottleneck

In most cases Redis performance is limited by:

```
network I/O
```

not CPU.

So multi-threading wouldn't improve much.

3. Redis Event Loop

Redis uses a **non-blocking event loop** similar to:

- Node.js
- Nginx

It is built on **epoll** / **kqueue** / **select** depending on OS.

Event loop workflow

```
while(server_running):  
  
    check_new_connections()  
  
    read_client_requests()  
  
    execute_commands()  
  
    write_responses()
```

This allows Redis to handle **100k+ connections**.

4. Redis Request Lifecycle

Let's trace a real request.

Client command:

```
SET user:1 "Arpit"
```

Flow inside Redis:

```
Client  
|  
TCP packet  
|
```

```
Socket
|
Event Loop
|
Command Parser
|
Command Execution
|
Memory Store
|
Response
```

5. Redis Memory Store

All Redis data is stored in RAM.

Internally Redis uses optimized data structures.

Examples:

Redis Type	Internal Structure
String	SDS (Simple Dynamic String)
List	QuickList
Hash	HashTable
Set	HashTable
Sorted Set	Skip List

6. Simple Dynamic Strings (SDS)

Redis does **NOT** use normal C strings.

Normal C string:

```
char *str
```

Problems:

- requires scanning for length
- buffer overflow risk

Redis uses:

```
struct SDS {
    len
    free
```

```
    buf[]  
}
```

Example:

```
len = 5  
free = 10  
buf = "hello"
```

Advantages:

- O(1) length lookup
- safe memory
- efficient resizing

7. Redis Object Model

Every value stored in Redis is wrapped in a Redis object.

```
redisObject  
|  
+---- type  
+---- encoding  
+---- reference count  
+---- pointer to data
```

Example:

```
SET name "arpit"
```

Redis creates:

```
redisObject(type=string)
```

8. Redis Command Execution

Every Redis command maps to a C function.

Example:

```
SET key value
```

calls:

```
setCommand()
```

Similarly:

```
GET key
```

calls:

```
getCommand()
```

Commands execute **atomically** because Redis is single threaded.

Meaning:

```
no two commands execute at same time
```

9. Redis Networking Layer

Redis communicates via **RESP protocol**

RESP = Redis Serialization Protocol

Example request:

```
*3
$3
SET
$4
name
$5
arpit
```

Response:

```
+OK
```

RESP supports:

- strings
 - integers
 - arrays
 - errors
 - bulk strings
-

10. Redis Performance

Typical performance:

Operation	Speed
GET	~100k ops/sec
SET	~100k ops/sec
Pipeline	~1M ops/sec

Latency:

~1 ms

11. Redis Memory Allocation

Redis manages memory using:

jemalloc

This helps:

- reduce fragmentation
 - faster allocation
 - stable performance
-

12. Redis Command Table

Internally Redis keeps a **command lookup table**.

Example:

```
SET -> setCommand
GET -> getCommand
INCR -> incrCommand
```

This allows **O(1) command lookup**.

13. Redis Key Space

All keys exist inside the **keyspace dictionary**.

```
keyspace = {
  "user:1": object,
  "session:22": object,
  "counter": object
}
```

Internally implemented using:

Hash Table

So lookup is:

O(1)

14. Why Redis Is Extremely Fast

Redis performance comes from:

1. In-memory storage
 2. Single-threaded execution
 3. Efficient data structures
 4. Event-driven networking
 5. Minimal overhead
-

Key Takeaway

Redis is essentially:

An in-memory data structure engine running on an event-driven single-threaded server optimized for extremely fast operations.

Next Topic

Redis Deep Dive — Part 2

We will study:

1. Redis Data Structures Internals
2. Skip Lists
3. QuickLists
4. HashTable rehashing
5. Sorted set implementation
6. Memory optimizations

This is where Redis becomes **really interesting**.

Redis Deep Dive — Part 2

Redis Data Structures Internals

Redis is extremely fast not just because it uses RAM, but because it uses **very carefully engineered internal data structures**.

Each Redis data type is implemented using **specialized optimized structures**.

1. Redis Data Structure Overview

Redis Type	Internal Implementation
String	SDS (Simple Dynamic String)
List	QuickList
Set	Hash Table

Hash	Hash Table / Ziplist
Sorted Set	Skip List + Hash Table
Streams	Radix Tree + Listpack

Redis dynamically changes internal structures based on **size and usage**.

This is called:

```
ENCODING OPTIMIZATION
```

2. Redis Strings (SDS)

Redis strings use **SDS — Simple Dynamic Strings**.

Why not normal C strings?

C strings:

```
char *str
```

Problems:

1. Length requires O(n)
2. Buffer overflow risks
3. Memory inefficiency

SDS Structure

```
struct sdshdr {  
    int len  
    int free  
    char buf[]  
}
```

Example:

```
len = 5  
free = 10  
buf = "hello"
```

Advantages:

- O(1) length lookup
 - binary safe
 - dynamic resizing
 - prevents buffer overflow
-

3. Redis Lists — QuickList

Redis originally used:

```
Linked List
```

But linked lists waste memory.

Then Redis introduced:

```
Ziplist
```

But ziplists had performance issues.

Modern Redis uses:

```
QuickList
```

QuickList Structure

A QuickList is:

```
LinkedList of Ziplists
```

Diagram:

```
QuickList
```

```
Node -> Ziplist -> elements
```

```
Node -> Ziplist -> elements
```

```
Node -> Ziplist -> elements
```

Example:

```
tasks = ["task1","task2","task3"]
```

Internally:

```
Node1 -> [task1 task2 task3]
```

Why QuickList is Efficient

Advantages:

1. Reduced pointer overhead
2. Better cache locality
3. Faster traversal

4. Lower memory usage

4. Redis Hash Tables

Used for:

- Sets
- Hashes
- Keyspace dictionary

Redis hash table structure:

```
dict
|
+-- hash table
+-- size
+-- used slots
```

Each entry:

```
dictEntry
|
+-- key
+-- value
+-- next (collision chain)
```

Collision Handling

Redis uses:

```
Separate chaining
```

Example:

```
hash(key) -> bucket

bucket -> entry -> entry -> entry
```

5. Incremental Rehashing

When a hash table grows, Redis must rehash.

Normal systems:

```
rehash everything at once
```

This causes:

```
latency spikes
```

Redis solution:

```
INCREMENTAL REHASHING
```

How It Works

Redis keeps **two hash tables temporarily**.

```
ht[0] -> old table  
ht[1] -> new table
```

Rehashing happens gradually.

Each command moves some keys.

Example:

```
SET key1  
→ move few entries
```

Result:

```
no long blocking operations
```

6. Redis Sets

Sets internally use:

```
Hash Table
```

Example:

```
SADD users arpit
```

Stored internally as:

```
users -> hash table
```

Lookup:

```
SISMEMBER users arpit
```

Complexity:

$O(1)$

7. Redis Sorted Sets (ZSET)

Sorted sets are one of Redis's **most powerful structures**.

Example:

```
ZADD leaderboard 100 arpit
```

Internally Redis uses **two structures**.

```
Hash Table  
+  
Skip List
```

Why Two Structures?

Hash Table:

```
member -> score
```

Skip List:

```
sorted by score
```

This enables:

- fast lookup
- fast ranking
- fast range queries

8. Skip Lists

Skip lists are an alternative to balanced trees.

Structure:

```
Layer 3:      -----> 100  
Layer 2:  ----> 50 ----> 100  
Layer 1:  10 -> 20 -> 50 -> 70 -> 100
```

Search complexity:

$O(\log n)$

Advantages over trees:

- simpler implementation
 - good performance
 - easier concurrency (though Redis is single threaded)
-

9. Redis Hashes

Hashes store objects.

Example:

```
HSET user:1 name Arpit
HSET user:1 age 22
```

Internally Redis may use:

```
Ziplist / Listpack
```

for small hashes.

Example internal storage:

```
[name, Arpit, age, 22]
```

Advantages:

- compact memory usage
- faster for small objects

If hash grows large:

```
convert to hash table
```

10. Redis Memory Optimization

Redis automatically changes encoding.

Example:

Small hash:

```
Ziplist
```

Large hash:

```
HashTable
```

Small sorted set:

Ziplist

Large sorted set:

SkipList

This optimization dramatically reduces memory.

11. Redis Streams (Brief)

Redis Streams are used for:

- event logs
- messaging
- Kafka-like workloads

Internally streams use:

Radix Tree
+
Listpack

Radix tree helps with:

ordered message storage

12. Time Complexity of Redis Operations

Operation	Complexity
GET	O(1)
SET	O(1)
LPUSH	O(1)
SADD	O(1)
ZADD	O(log N)
ZRANGE	O(log N + M)

Key Insight

Redis is not just:

Key → Value

It is actually:

Key → Optimized Data Structure

Which is why Redis can power:

- real time analytics
- leaderboards
- message queues
- rate limiting
- caching

Next Topic

Redis Deep Dive — Part 3

We will cover **Redis Persistence**.

1. RDB Snapshots
2. AOF (Append Only File)
3. Hybrid persistence
4. Crash recovery
5. Disk I/O optimization
6. fsync policies

This is where Redis becomes a **real database**.

Redis Deep Dive — Part 3

Redis Persistence

Redis is an **in-memory database**, meaning all data is stored in RAM.

But RAM is **volatile**.

If the server crashes:

all memory is lost

To solve this, Redis provides **persistence mechanisms** that save data to disk.

Redis supports three persistence approaches:

Method	Description
RDB	Periodic snapshots

AOF	Log of every write operation
Hybrid	Combination of RDB + AOF

1. RDB Persistence (Snapshotting)

RDB stands for:

Redis Database Snapshot

Redis periodically saves the entire dataset to disk.

Example file:

dump.rdb

How RDB Works

Redis performs a **point-in-time snapshot** of memory.

Flow:

```
Redis Memory
  |
  v
fork()
  |
  v
Child Process
  |
  v
Write snapshot to disk
```

Important:

The main Redis process continues serving requests

So Redis **does not block**.

Fork Mechanism

When snapshotting begins:

```
fork()
```

creates a child process.

Memory uses:

```
Copy-On-Write (COW)
```

Meaning:

- parent and child share memory pages
- only modified pages are copied

This makes snapshotting **very efficient**.

RDB Configuration

In `redis.conf` :

```
save 900 1
save 300 10
save 60 10000
```

Meaning:

Condition	Snapshot
1 change in 900 sec	Save
10 changes in 300 sec	Save
10000 changes in 60 sec	Save

Advantages of RDB

- Very fast recovery
 - Compact file
 - Minimal runtime overhead
 - Good for backups
-

Disadvantages of RDB

Data loss window.

Example:

```
snapshot every 5 minutes
```

If Redis crashes:

```
last 5 minutes of data may be lost
```

2. AOF Persistence (Append Only File)

AOF logs **every write command**.

Example file:

```
appendonly.aof
```

Instead of storing data snapshots, Redis stores **operations**.

Example commands:

```
SET user:1 "Arpit"  
INCR visits  
LPUSH tasks task1
```

These commands are appended to the AOF file.

AOF Workflow

```
Client Command  
  |  
  v  
Execute in Redis  
  |  
  v  
Append command to AOF file
```

On restart:

```
Redis replays the commands
```

to rebuild memory.

Example AOF File

```
SET name "arpit"  
SET age 22  
INCR visits  
LPUSH tasks "task1"
```

Redis replays them sequentially.

3. AOF fsync Policies

Disk writes are expensive.

Redis allows control over durability vs performance.

Configuration:

```
appendfsync always
appendfsync everysec
appendfsync no
```

Always

```
appendfsync always
```

Behavior:

- sync to disk after every command

Pros:

- safest

Cons:

- slow
-

Every Second (Recommended)

```
appendfsync everysec
```

Behavior:

- fsync every second

Pros:

- good durability
- good performance

Data loss risk:

```
~1 second
```

No

```
appendfsync no
```

Behavior:

- OS decides when to flush

Pros:

- fastest

Cons:

- risk of data loss
-

4. AOF Rewrite (Log Compaction)

Over time AOF grows large.

Example:

```
INCR counter
INCR counter
INCR counter
```

AOF might contain:

```
SET counter 1
INCR counter
INCR counter
INCR counter
```

But the final value is:

```
counter = 4
```

So Redis rewrites the log.

AOF Rewrite Process

Redis generates a **new compact AOF**.

Example rewritten file:

```
SET counter 4
```

This process:

```
does not block Redis
```

because it uses **background rewrite**.

5. Hybrid Persistence (Best Option)

Modern Redis uses:

```
RDB + AOF
```

Hybrid AOF file structure:

```
[RDB snapshot]
[recent commands]
```

Benefits:

- fast startup
- minimal data loss
- smaller files

6. Redis Crash Recovery

On restart Redis loads data.

Startup process:

```
Check AOF
|
|-- exists → load AOF
|
|-- else → load RDB
```

AOF has priority because:

```
it is more durable
```

7. Persistence Tradeoffs

Feature	RDB	AOF
Recovery speed	Fast	Slower
File size	Small	Large
Durability	Lower	Higher
CPU overhead	Low	Higher

8. When to Use Each

RDB Only

Use when:

- Redis used mainly as cache
 - backups needed
 - fast restart required
-

AOF

Use when:

- Redis used as primary DB
 - minimal data loss required
-

Hybrid (Recommended)

Use when:

Redis powers production systems

9. Example Production Config

Typical production Redis config:

```
appendonly yes
appendfsync everysec
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb
```

Key Insight

Redis persistence is based on two ideas:

Snapshots
+
Operation Logs

Exactly the same concepts used in:

- databases
 - distributed systems
 - Kafka
 - event sourcing systems
-

Next Topic

Redis Deep Dive — Part 4

We will cover:

1. Redis Replication
2. Master-Replica architecture
3. Partial resynchronization
4. Replication backlog
5. Failover

This is where Redis becomes a **distributed system**.

Redis Deep Dive — Part 4

Redis Replication & High Availability

Redis replication allows **multiple Redis servers to synchronize data**.

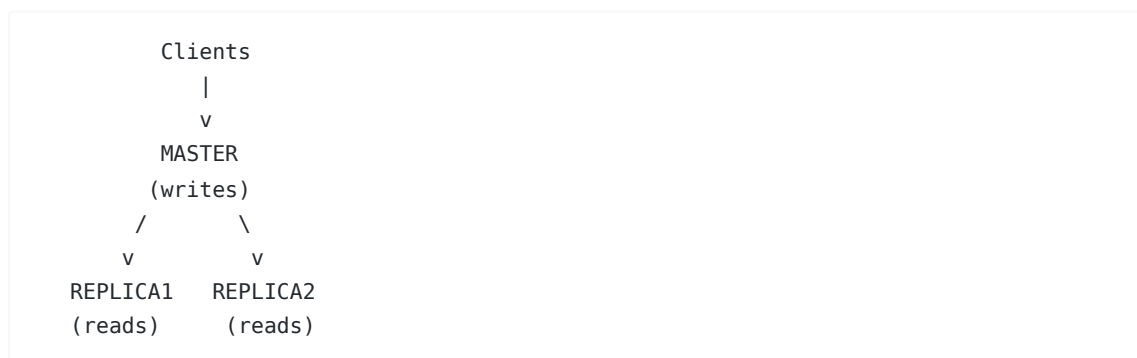
This enables:

- high availability
- read scaling
- backup nodes
- failover systems

1. Redis Replication Architecture

Redis replication uses a **master-replica architecture**.

Diagram:



Rules:

- **Writes go to master**
 - **Replicas copy master data**
 - Replicas are usually **read-only**
-

2. Setting Up Replication

To make a Redis server a replica:

```
replicaof <master-ip> <port>
```

Example:

```
replicaof 192.168.1.10 6379
```

Now the replica will start syncing with the master.

3. Initial Synchronization

When a replica connects to a master:

Steps:

1. Replica connects to master
2. Replica sends SYNC request
3. Master generates RDB snapshot
4. Snapshot sent to replica
5. Replica loads snapshot
6. Live commands streamed

Diagram:

```
MASTER MEMORY
  |
  v
Generate RDB Snapshot
  |
  v
Send to Replica
  |
  v
Replica loads snapshot
```

This is called:

```
FULL RESYNCHRONIZATION
```

4. Continuous Replication

After sync, replication becomes **stream-based**.

Every write command on master is sent to replicas.

Example:

Client command:

```
SET user:1 "arpit"
```

Master executes:

```
SET user:1 "arpit"
```

Then sends the same command to replicas.

5. Replication Backlog

Network failures may interrupt replication.

Example:

```
Replica disconnects for 5 seconds
```

Instead of full sync, Redis uses:

```
PARTIAL RESYNC
```

Redis maintains a **replication backlog buffer**.

Structure:

```
Master
|
Replication Buffer
|
Recent Commands
```

If replica reconnects quickly:

```
only missing commands are sent
```

This is much faster.

6. Replication IDs

Each Redis master has:

```
Replication ID
```

Example:

```
replication_id = abc123
```

Replica tracks:

```
replication offset
```

Example:

```
offset = 452000
```

When reconnecting:

```
send missing commands from offset
```

7. Asynchronous Replication

Redis replication is **asynchronous**.

Meaning:

```
master does not wait for replicas
```

Example flow:

```
Client → Master write  
Master executes immediately  
Master later sends update to replicas
```

Pros:

- very fast writes

Cons:

```
replicas may lag behind
```

8. Read Scaling

Because replicas hold full copies of data:

They can serve **read traffic**.

Architecture:

```
Clients
|
+---- Master (writes)
|
+---- Replica1 (reads)
|
+---- Replica2 (reads)
```

Benefits:

- reduces load on master
- improves scalability

9. Redis Failover Problem

If the master crashes:

```
system becomes unavailable
```

Because:

- replicas are read-only
- no automatic promotion

This problem is solved by:

```
Redis Sentinel
```

or

```
Redis Cluster
```

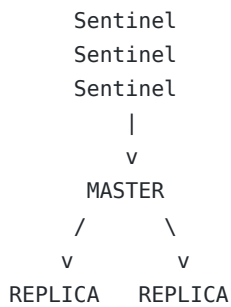
10. Redis Sentinel

Sentinel provides **automatic failover**.

Responsibilities:

- ```
1. Monitor Redis nodes
2. Detect failures
3. Promote replicas
4. Notify clients
```

Architecture:



---

## 11. Failover Process

If master crashes:

Steps:

1. Sentinel detects failure
2. Sentinels vote
3. One replica promoted to master
4. Other replicas sync to new master
5. Clients reconnect

Diagram:

```
Old Master (DOWN)

Replica1 → Promoted to Master

Replica2 → Becomes Replica of Replica1
```

---

## 12. Sentinel Quorum

To prevent false failovers:

Multiple sentinels must agree.

Example:

```
quorum = 3
```

At least **3 sentinels must detect failure.**

---

## 13. Replication Latency

Replication delay depends on:

- network latency
- replication backlog size
- write throughput

Typical delay:

milliseconds

---

## 14. Redis Replication Limitations

Important limitations:

### No strong consistency

Because replication is async.

Example:

```
write success on master
replica still outdated
```

---

### Writes only on master

Redis does not support:

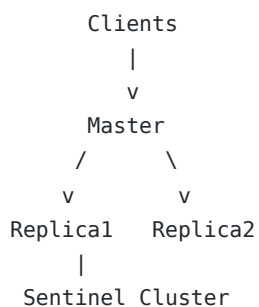
```
multi-master replication
```

(Except with special setups.)

---

## 15. Production Redis Architecture

Typical production architecture:



---

## Key Insight

Redis replication works by **streaming commands from master to replicas**.

Key principles:

Snapshot + Command Stream

Exactly the same idea used in:

- MySQL replication
- Kafka log replication
- event sourcing systems

---

## Next Topic

Redis Deep Dive — Part 5

We will study **Redis Cluster (Sharding)**.

Topics:

1. Horizontal scaling
2. Hash slots
3. Key distribution
4. Cluster communication
5. Resharding
6. Gossip protocol

This is the **real large-scale Redis architecture used by companies**.

## Redis Deep Dive — Part 5

### Redis Cluster (Horizontal Scaling)

A single Redis instance has limits:

- limited RAM
- limited CPU
- single machine failure risk

To scale beyond one machine, Redis uses **Redis Cluster**.

Cluster enables:

Horizontal scaling  
+  
High availability  
+  
Automatic sharding

---

## 1. What is Sharding?

Sharding means **splitting data across multiple servers**.

Example:

```
User Data
```

Instead of storing everything on one server:

```
Server1 → users 1–1M
Server2 → users 1M–2M
Server3 → users 2M–3M
```

Each server stores **a subset of data**.

---

## 2. Redis Cluster Architecture

Redis Cluster divides data across nodes.

Example cluster:

```
Client
|
v
+-----+
| Redis Cluster |
+-----+
| | |
v v v
Node1 Node2 Node3
```

Each node stores **different keys**.

---

## 3. Hash Slots (Core Idea)

Redis Cluster does not shard by key ranges.

Instead it uses **hash slots**.

Total slots:

```
16384 hash slots
```

Every key maps to **one slot**.

Example:

```
hash(key) % 16384 = slot
```

---

## 4. Slot Distribution

Example cluster:

```
Node1 → slots 0–5000
Node2 → slots 5001–10000
Node3 → slots 10001–16383
```

Key routing:

```
SET user:1
```

Compute:

```
CRC16("user:1") % 16384
```

Example result:

```
slot = 7342
```

Stored on:

```
Node2
```

---

## 5. Client Routing

Redis clients must know the cluster topology.

Workflow:

```
Client → any node
Node → responds with correct node
Client → sends request to correct node
```

Example response:

```
MOVED 7342 192.168.1.20:6379
```

Meaning:

```
slot 7342 is on another node
```

Client retries request.

---

## 6. Cluster Node Communication

Cluster nodes communicate using a **gossip protocol**.

Nodes periodically exchange information:

```
node health
slot ownership
replication state
cluster topology
```

Example communication:

```
PING
PONG
FAIL
MEET
```

This keeps the cluster **synchronized**.

---

## 7. Master and Replica Nodes

Each master node can have replicas.

Example cluster:

```
Master1 → Replica1
Master2 → Replica2
Master3 → Replica3
```

Diagram:

```
Master1
|
Replica1

Master2
|
Replica2

Master3
|
Replica3
```

Replicas provide **failover support**.

---

## 8. Automatic Failover

If a master node fails:

```
Replica promoted → new master
```

Process:

1. Cluster detects failure
2. Replicas vote
3. One replica promoted
4. Slots reassigned

Example:

```
Master2 crashes
Replica2 becomes Master2
```

Cluster continues operating.

---

## 9. Resharding (Moving Data)

When new nodes are added:

```
cluster reshard
```

Slots are redistributed.

Example:

Before:

```
Node1 → 8000 slots
Node2 → 8000 slots
```

After adding Node3:

```
Node1 → 5500
Node2 → 5500
Node3 → 5500
```

Redis migrates keys gradually.

---

## 10. Hash Tags

Sometimes multiple keys must be on the same node.

Example:

```
user:1
user:1:orders
user:1:cart
```

Use **hash tags**.

Example:

```
{user1}:profile
{user1}:orders
{user1}:cart
```

Redis hashes only:

```
user1
```

So all keys land on the **same slot**.

---

## 11. Multi-Key Operations Limitation

Cluster limitation:

Multi-key commands must use **same slot**.

Example invalid operation:

```
MGET user:1 product:2
```

Because:

```
different slots
```

Solution:

```
hash tags
```

---

## 12. Cluster Failure Detection

Cluster nodes constantly check each other.

States:

PFAIL → Possibly failing  
FAIL → Confirmed failure

If enough nodes agree:

node marked as FAIL

Failover begins.

---

## 13. Redis Cluster Characteristics

| Feature            | Redis Cluster |
|--------------------|---------------|
| Sharding           | Yes           |
| Automatic failover | Yes           |
| Replication        | Yes           |
| Horizontal scaling | Yes           |
| Consistency        | Eventual      |

---

## 14. Typical Production Redis Cluster

Example production setup:

3 Master Nodes  
3 Replica Nodes

Architecture:

Master1 → Replica1  
Master2 → Replica2  
Master3 → Replica3

Total nodes:

6 nodes

This provides:

- redundancy
  - scaling
  - fault tolerance
-

## 15. Redis Cluster Limitations

Important limitations:

### No multi-key cross slot transactions

Example:

```
MSET key1 key2
```

Only allowed if:

```
same slot
```

---

### Slightly higher latency

Because client may need:

```
redirection
```

---

## Key Insight

Redis Cluster works using:

```
Consistent hashing
+
Slot distribution
+
Gossip protocol
+
Replica failover
```

This design allows Redis to scale to **terabytes of data**.

---

## Next Topic

Redis Deep Dive — Part 6

We will cover **Redis Performance Engineering**.

Topics:

1. Redis pipelining
2. Redis transactions
3. Lua scripting
4. Redis memory eviction

5. LRU / LFU caching
6. Performance tuning

These are the techniques used by **high performance systems**.

## Redis Deep Dive — Part 6

### Redis Performance Engineering

Redis is already extremely fast, but with proper techniques it can handle:

```
millions of operations per second
```

Key performance techniques:

1. Pipelining
2. Transactions
3. Lua scripting
4. Memory eviction policies
5. Efficient key design
6. Connection management

---

## 1. Redis Pipelining

Normally Redis processes commands sequentially.

Example without pipelining:

```
Client → SET key1 value
wait response

Client → SET key2 value
wait response

Client → SET key3 value
wait response
```

Problem:

```
network round-trip time (RTT)
```

dominates performance.

---

### Pipelining Concept

Pipelining allows sending **multiple commands without waiting for responses**.

Example:

Client sends:

```
SET key1 value
SET key2 value
SET key3 value
SET key4 value
SET key5 value
```

Redis processes them sequentially but responses are returned together.

---

## Performance Impact

Without pipeline:

```
100k ops/sec
```

With pipeline:

```
1M+ ops/sec
```

Pipelining drastically reduces network overhead.

---

## Python Example

```
import redis

r = redis.Redis()

pipe = r.pipeline()

pipe.set("key1", "value1")
pipe.set("key2", "value2")
pipe.set("key3", "value3")

pipe.execute()
```

## 2. Redis Transactions

Redis supports transactions using:

```
MULTI
EXEC
```

Example:

```
MULTI
SET balance 100
INCR visits
EXEC
```

All commands execute **atomically**.

Meaning:

```
no other client can interleave commands
```

---

## Important Limitation

Redis transactions do **not support rollbacks**.

Example:

```
MULTI
SET a 10
INVALID COMMAND
EXEC
```

Redis will execute valid commands and ignore invalid ones.

---

## 3. WATCH (Optimistic Locking)

Redis supports optimistic concurrency control.

Example:

```
WATCH balance
```

If the key changes before transaction:

```
EXEC fails
```

Example workflow:

```
WATCH balance
GET balance

MULTI
SET balance new_value
EXEC
```

If another client modifies the key:

EXEC → aborts

---

## 4. Lua Scripting

Redis allows running **Lua scripts inside the server**.

Benefits:

- atomic execution
- reduces network calls
- extremely fast

Example:

```
EVAL "
local current = redis.call('GET', KEYS[1])
if current then
 return redis.call('INCR', KEYS[1])
else
 return 0
end
" 1 counter
```

Redis executes the script **atomically**.

---

## Why Lua Scripts Are Powerful

Instead of:

```
GET counter
INCR counter
```

You can run logic **inside Redis**.

This reduces:

```
network latency
```

---

## 5. Redis Memory Eviction

When Redis memory is full:

```
new writes fail
```

Unless eviction is enabled.

Configuration:

```
maxmemory 2gb
maxmemory-policy allkeys-lru
```

## 6. Eviction Policies

Redis supports several eviction strategies.

| Policy          | Description                    |
|-----------------|--------------------------------|
| noeviction      | return error                   |
| allkeys-lru     | remove least recently used key |
| volatile-lru    | remove LRU keys with TTL       |
| allkeys-random  | random key eviction            |
| volatile-random | random TTL key                 |
| allkeys-lfu     | least frequently used          |
| volatile-lfu    | LFU keys with TTL              |

### LRU (Least Recently Used)

Removes keys **not used recently**.

Example:

```
cache:product:1
cache:product:2
cache:product:3
```

Least accessed key gets removed first.

### LFU (Least Frequently Used)

Removes keys accessed **least often**.

Example:

```
productA accessed 100 times
productB accessed 2 times
```

Redis evicts:

```
productB
```

## 7. Redis Key Design

Bad key design causes:

- memory waste
- slower lookups

Good pattern:

```
object:id:field
```

Example:

```
user:1001:name
user:1001:email
cart:1001:items
```

This improves:

- readability
  - debugging
  - maintenance
- 

## 8. Redis Connection Management

Opening connections repeatedly is expensive.

Bad pattern:

```
connect
command
disconnect
```

Correct approach:

```
connection pool
```

Example:

```
pool = redis.ConnectionPool(host='localhost', port=6379)
r = redis.Redis(connection_pool=pool)
```

---

## 9. Redis Hot Keys Problem

A **hot key** is accessed extremely frequently.

Example:

```
GET homepage_cache
```

Millions of requests.

Problem:

```
single node overload
```

Solutions:

```
replication
client-side caching
key sharding
```

---

## 10. Redis Big Key Problem

Big keys contain too much data.

Example:

```
SET huge_json 10MB
```

Problems:

- slow transfer
- memory spikes
- blocking operations

Solutions:

```
split keys
use hashes
store references
```

---

## 11. Redis Latency Monitoring

Redis provides tools:

```
LATENCY DOCTOR
```

Example:

```
LATENCY LATEST
```

Shows:

- slow commands
- blocking operations

---

## 12. Redis Benchmarking

Testing Redis performance:

```
redis-benchmark
```

Example:

```
redis-benchmark -n 100000 -c 50
```

Parameters:

| Flag | Meaning            |
|------|--------------------|
| -n   | total requests     |
| -c   | concurrent clients |

---

## Key Insight

Redis performance depends more on:

```
network round trips
```

than raw CPU.

So the biggest optimizations are:

```
pipelining
Lua scripting
efficient key design
```

---

## Next Topic

Redis Deep Dive — Part 7

We will explore **Redis Advanced Systems Use Cases**:

1. Distributed locks
2. Rate limiting
3. Message queues
4. Redis Streams
5. Leaderboards

6. Real-time analytics

These are the **actual production patterns used by large systems**.

## Redis Deep Dive — Part 7

### Redis System Design Patterns

Redis is often used as an **infrastructure component** rather than just a database.

Common production patterns:

1. Distributed Locks
2. Rate Limiting
3. Message Queues
4. Redis Streams
5. Leaderboards
6. Real-time analytics
7. Caching patterns

---

## 1. Distributed Locks

In distributed systems, multiple servers may try to access the same resource.

Example:

```
Server A → process order
Server B → process order
```

Without locking:

```
duplicate processing
```

Redis provides distributed locking.

---

### Basic Lock Implementation

Command:

```
SET lock_key value NX PX 30000
```

Meaning:

| Option | Meaning                        |
|--------|--------------------------------|
| NX     | only set if key does not exist |
| PX     | expiration time                |

Example:

```
SET order_lock "serverA" NX PX 10000
```

If successful:

```
lock acquired
```

---

## Unlocking Safely

Unlock using Lua script:

```
if redis.call("GET", KEYS[1]) == ARGV[1] then
 return redis.call("DEL", KEYS[1])
else
 return 0
end
```

This ensures **only the lock owner releases it**.

---

## 2. Rate Limiting

Used to prevent API abuse.

Example:

```
100 requests per minute per user
```

---

### Simple Rate Limiter

```
INCR api:user123
EXPIRE api:user123 60
```

Workflow:

```
if count > 100 → block request
```

---

### Sliding Window Rate Limiter

More precise rate limiting.

Example:

```
ZADD requests timestamp request_id
ZREMRANGEBYSCORE requests 0 old_timestamp
ZCARD requests
```

Redis sorted sets store request timestamps.

---

### 3. Message Queue

Redis can function as a **lightweight message broker**.

Example:

```
LPUSH jobs job1
```

Worker:

```
BRPOP jobs
```

Flow:

```
Producer → Redis Queue → Worker
```

---

### Queue Architecture

```
Producer
 |
 v
Redis List
 |
 v
Worker1
Worker2
Worker3
```

Multiple workers can process jobs.

---

### 4. Redis Streams (Kafka-like)

Streams support **event streaming systems**.

Example:

```
XADD events * user_id 100 action login
```

Stored as:

```
event_id → fields
```

Example event:

```
1678200000-0
user_id = 100
action = login
```

---

## Consumer Groups

Streams support **multiple consumers**.

Example:

```
XGROUP CREATE events group1 0
```

Consumers read:

```
XREADGROUP GROUP group1 consumer1 COUNT 1 STREAMS events >
```

Benefits:

- parallel processing
- reliable delivery
- event replay

---

## 5. Leaderboards

Redis sorted sets power **leaderboards**.

Example:

```
ZADD leaderboard 100 userA
ZADD leaderboard 200 userB
```

Top players:

```
ZREVRANGE leaderboard 0 9 WITHSCORES
```

Player rank:

```
ZRANK leaderboard userA
```

Used in:

- gaming
- ranking systems

- analytics

---

## 6. Real-Time Analytics

Redis can track real-time metrics.

Example:

```
INCR page_views
```

Or per user:

```
INCR page_views:user123
```

Time-based metrics:

```
pageviews:2026-03-17
```

---

## 7. Caching Patterns

Redis is commonly used as a **cache layer**.

Architecture:

```
Client
|
v
Redis Cache
|
v
Database
```

---

### Cache Aside Pattern

Flow:

1. Check Redis
2. If miss → query DB
3. Store result in Redis
4. Return response

Example:

```
GET user:1
```

If missing:

```
query database
SET user:1 result EX 3600
```

---

## Write Through Cache

Flow:

```
write to cache
write to database
```

Ensures cache consistency.

---

## Write Back Cache

Flow:

```
write to cache
database updated later
```

Used in **high throughput systems**.

---

# 8. Session Store

Redis is often used for **user sessions**.

Example:

```
SET session:abc123 user_data EX 3600
```

Advantages:

- fast lookup
  - automatic expiration
  - scalable
- 

# 9. Pub/Sub Messaging

Redis supports **publish-subscribe messaging**.

Publisher:

```
PUBLISH chat "hello"
```

Subscriber:

```
SUBSCRIBE chat
```

Use cases:

- notifications
- chat systems
- live updates

---

## 10. Redis in Large Architectures

Typical modern architecture:

```
Users
|
v
API Gateway
|
v
Application Servers
|
+---- Redis Cache
|
+---- Redis Queue
|
+---- Redis Streams
|
+---- Database
```

Redis often becomes a **central infrastructure component**.

---

## Key Insight

Redis is used not just as a database but as:

```
Cache
Queue
Lock manager
Stream processor
Real-time analytics engine
```

---

## Final Redis Knowledge Stack

To master Redis you should understand:

1. Redis architecture
2. Redis data structures

3. Persistence
4. Replication
5. Redis Cluster
6. Performance optimization
7. System design patterns

Once you understand these, you can design **high performance distributed systems using Redis**.

---

## What to Explore Next

Advanced Redis topics:

- Redis modules
- Redis AI
- Redis Bloom filters
- Redis time-series
- Redis graph database
- Redis vector search

## Redis Deep Dive — Part 8

### Redis in Modern AI Systems

Modern AI platforms use Redis as a **real-time infrastructure layer**.

Redis is used for:

1. Vector databases
2. Agent memory
3. AI pipelines
4. Task orchestration
5. Feature stores
6. Real-time analytics

---

## 1. Redis as a Vector Database

AI systems need to store **embeddings**.

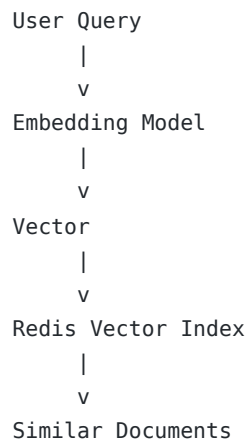
Example embedding:

```
[0.12, 0.44, 0.98, ...]
```

Redis can store vectors using:

```
Redis Vector Similarity Search
```

Architecture:



---

## Example Use Case

Semantic search.

Example:

User asks:

"best restaurants in Bangalore"

Steps:

1. Convert query → embedding
2. Search vector index
3. Retrieve similar documents

---

## Redis Vector Query

Example:

```
FT.SEARCH idx "*"=>[KNN 10 @vector $query_vector]"
```

Returns the **closest vectors**.

---

## 2. Redis as Agent Memory

AI agents need **short-term memory**.

Example memory:

User: prefers vegetarian food  
User: lives in Bangalore

Store in Redis:

```
SET agent:user123:memory {...}
```

Workflow:

```
graph TD
 A[User Query] --> B[Agent retrieves memory]
 B --> C[LLM generates response]
```

Redis provides **fast retrieval**.

---

## 3. Conversation Memory

Chat systems often store conversation history.

Example:

```
LPUSH chat:user123 message
```

Retrieve last messages:

```
LRange chat:user123 0 10
```

Used by:

- ChatGPT-like systems
  - AI assistants
  - support chatbots
- 

## 4. AI Task Queues

AI workloads often require background processing.

Example tasks:

- embedding generation
- summarization
- indexing documents
- model inference

Redis queues handle these tasks.

Architecture:

```
Client
 |
 v
Redis Queue
 |
 v
Worker Nodes
```

Example:

```
LPUSH tasks embedding_job
```

Worker:

```
BRPOP tasks
```

---

## 5. AI Pipeline Orchestration

Complex AI systems use pipelines.

Example pipeline:

```
Document Upload
 |
 v
Embedding Generation
 |
 v
Vector Storage
 |
 v
Search Index
```

Redis Streams coordinate pipeline stages.

Example:

```
XADD pipeline doc_id 123 stage embedding
```

Workers consume tasks.

---

## 6. Redis Streams for AI Pipelines

Streams support **event-driven AI pipelines**.

Example architecture:

```
Producer → Redis Stream → AI Workers
```

Example:

```
XADD ai_pipeline * step embedding doc_id 10
```

Consumers process:

```
embedding worker
indexing worker
summarization worker
```

---

## 7. Feature Store for ML

Machine learning models require **feature storage**.

Example features:

```
user_age
user_click_rate
purchase_history
```

Store in Redis:

```
HSET features:user123 age 25 click_rate 0.42
```

Advantages:

- fast retrieval
- real-time updates
- scalable

---

## 8. Real-Time Recommendation Systems

Example recommendation architecture:

```
User Action
 |
 v
Event Stream
 |
 v
Redis
 |
```

```
v
Recommendation Engine
```

Redis stores:

- user vectors
- item vectors
- interaction history

---

## 9. AI Rate Limiting

LLM APIs often require rate limits.

Example:

```
100 requests per minute per user
```

Redis implementation:

```
INCR llm:user123
EXPIRE llm:user123 60
```

---

## 10. Distributed AI Systems

Large AI systems require distributed infrastructure.

Example architecture:

```
Users
|
v
API Gateway
|
v
AI Orchestrator
|
+---- Redis Cache
|
+---- Redis Queue
|
+---- Redis Streams
|
+---- Vector Database
|
+---- GPU Workers
```

Redis acts as **the coordination layer**.

---

# 11. Redis for Agent Frameworks

Agent frameworks (LangChain, LangGraph) use Redis for:

- state storage
- memory
- queueing tasks
- tool execution logs

Example:

```
agent:state:task123
```

Redis enables **multi-agent coordination**.

# 12. Redis vs Traditional Databases for AI

| Feature             | Redis        | SQL DB       |
|---------------------|--------------|--------------|
| Latency             | microseconds | milliseconds |
| Vector search       | yes          | limited      |
| Real-time pipelines | yes          | limited      |
| Event streaming     | yes          | limited      |

Redis is optimized for **real-time AI workloads**.

# Key Insight

Modern AI systems often rely on Redis because it provides:

```
ultra-low latency
real-time data structures
event streaming
distributed coordination
```

Redis often becomes the **central nervous system** of AI infrastructure.

# Final Redis Mastery Roadmap

To truly master Redis:

1. Internal architecture
2. Data structures
3. Persistence

4. Replication
5. Redis cluster
6. Performance engineering
7. System design patterns
8. AI infrastructure use cases

Once you understand these, Redis becomes a **powerful system design tool**, not just a database.

## Redis Deep Dive — Part 9

### Real Production Architectures Using Redis

Many large tech companies rely on Redis as a **core infrastructure component**.

Companies using Redis extensively:

- Uber
- Netflix
- Discord
- Twitter/X
- Pinterest
- Shopify
- GitHub

Redis is usually not the main database but acts as a **high-speed infrastructure layer**.

---

## 1. Uber Architecture

Uber handles millions of real-time requests:

- ride matching
- location updates
- surge pricing
- trip tracking

Redis is used for **real-time geospatial queries and caching**.

---

### Uber Ride Matching

Architecture:

```
Driver App → API Gateway
 |
 v
 Redis Geo Index
 |
 v
 Nearby Drivers
```

Redis stores driver locations using:

```
GEOADD drivers longitude latitude driver_id
```

Example:

```
GEOADD drivers 77.5946 12.9716 driver_123
```

Find nearby drivers:

```
GEORADIUS drivers 77.5946 12.9716 5 km
```

Redis allows Uber to find nearby drivers **in milliseconds**.

---

## 2. Netflix Architecture

Netflix uses Redis for:

- caching metadata
- session storage
- real-time personalization

Architecture:

```
Users
 |
 v
API Gateway
 |
 v
Microservices
 |
+---- Redis Cache
 |
+---- Cassandra Database
```

Example:

User opens Netflix homepage.

Workflow:

1. Check Redis cache
2. If hit → return recommendations
3. If miss → query database
4. Store in Redis

This reduces **database load massively**.

---

## 3. Discord Architecture

Discord supports **millions of real-time chat messages**.

Redis is used for:

- message queues
- pub/sub messaging
- session storage

---

### Discord Messaging Flow

Architecture:

```
graph TD
 User[User] -- Message --> Gateway[Gateway]
 Gateway -- Server --> Redis[Redis]
 Redis -- Pub/Sub --> Chat[Chat Servers]
```

Example:

Publisher:

```
PUBLISH channel:123 "hello world"
```

Subscribers receive messages instantly.

Redis provides **low latency real-time messaging**.

---

## 4. Twitter/X Architecture

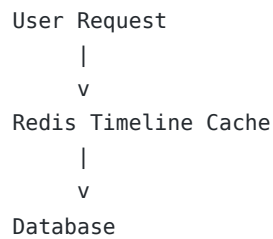
Twitter uses Redis for:

- timeline caching
- rate limiting
- trending analytics

---

### Timeline Caching

Architecture:



Example cache key:

```
timeline:user123
```

When user opens Twitter:

```
GET timeline:user123
```

This prevents expensive DB queries.

---

## 5. Pinterest Architecture

Pinterest uses Redis for **recommendation systems**.

Redis stores:

- user features
- content embeddings
- ranking scores

Example:

```
ZADD ranking:user123 score pin_456
```

Get top pins:

```
ZREVRANGE ranking:user123 0 20
```

Sorted sets power **real-time ranking systems**.

---

## 6. Shopify Architecture

Shopify handles massive e-commerce traffic.

Redis is used for:

- session storage
- product caching
- rate limiting

Example session:

```
SET session:user123 session_data EX 3600
```

This allows Shopify to support **high traffic stores**.

---

## 7. GitHub Architecture

GitHub uses Redis for:

- caching repository metadata
- job queues
- background workers

Example job queue:

```
LPUSH jobs build_repo
```

Workers process jobs asynchronously.

---

## 8. Cloudflare Architecture

Cloudflare uses Redis for **rate limiting and security**.

Example:

```
INCR ip:192.168.1.10
EXPIRE ip:192.168.1.10 60
```

If requests exceed limit:

```
block IP
```

Redis enables **real-time DDoS protection**.

---

## 9. Common Redis Patterns in Big Tech

Across companies, Redis is commonly used for:

| Use Case      | Example         |
|---------------|-----------------|
| Caching       | API responses   |
| Session Store | user sessions   |
| Rate Limiting | API throttling  |
| Queues        | background jobs |

|              |                     |
|--------------|---------------------|
| Pub/Sub      | real-time messaging |
| Leaderboards | rankings            |
| Geo Queries  | location services   |

## 10. Typical Modern Architecture with Redis

Example production system:

```

Users
 |
 v
Load Balancer
 |
 v
API Gateway
 |
 v
Application Servers
 |
 +---- Redis Cache
 |
 +---- Redis Queue
 |
 +---- Redis Streams
 |
 +---- Primary Database

```

Redis becomes the **high-speed middle layer**.

## 11. Why Companies Choose Redis

Redis advantages:

```

microsecond latency
high throughput
rich data structures
simple deployment
horizontal scalability

```

These properties make Redis ideal for **real-time systems**.

## 12. Redis at Massive Scale

Large companies run **huge Redis clusters**.

Example:

```
hundreds of Redis nodes
terabytes of RAM
millions of ops/sec
```

Redis clusters often support:

- global services
- AI pipelines
- real-time analytics

---

## Key Insight

Redis is rarely used as a standalone database.

Instead it acts as:

```
the high-speed infrastructure layer
between applications and databases
```

This architecture enables **massive scalability**.

---

## Redis Knowledge Summary

You now understand:

1. Redis architecture
2. Redis data structures
3. Persistence mechanisms
4. Replication
5. Redis cluster
6. Performance engineering
7. System design patterns
8. AI infrastructure usage
9. Real-world architectures

With this knowledge you can design **production-grade distributed systems using Redis**.

## Redis Deep Dive — Part 10

## Advanced System Design Problems Solved Using Redis

Redis is often used to solve **distributed system problems**.

Below are common system design problems where Redis is the best tool.

---

## 1. Global Rate Limiting

Problem:

Limit API usage globally.

Example:

```
100 requests per minute per user
```

Redis solution:

```
INCR api:user123
EXPIRE api:user123 60
```

Flow:

```
Client Request
 |
 v
Redis Counter
 |
 v
Allow / Reject Request
```

Used by:

- API gateways
- payment systems
- authentication services

---

## 2. Distributed Locking

Problem:

Multiple services try to update the same resource.

Example:

```
two services processing the same order
```

Redis solution:

```
SET lock:order123 serverA NX PX 10000
```

Meaning:

- only one service gets the lock

- lock expires automatically

Used in:

- payment systems
  - inventory systems
  - job schedulers
- 

## 3. Distributed Job Queue

Problem:

Handle background jobs asynchronously.

Example:

```
send emails
generate reports
process uploads
```

Redis solution:

```
LPUSH jobs job_data
```

Workers consume jobs:

```
BRPOP jobs
```

Architecture:

```
Producer
 |
 v
Redis Queue
 |
 v
Worker Pool
```

---

## 4. Real-Time Leaderboards

Problem:

Maintain rankings in real time.

Example:

```
gaming leaderboard
top influencers
```

```
top sellers
```

Redis solution:

```
ZADD leaderboard 100 userA
ZADD leaderboard 200 userB
```

Top users:

```
ZREVRANGE leaderboard 0 10 WITHSCORES
```

Ranking:

```
ZRANK leaderboard userA
```

Time complexity:

```
O(log N)
```

---

## 5. Real-Time Chat Systems

Problem:

Deliver messages instantly to millions of users.

Redis solution:

```
Pub/Sub
```

Example:

Publisher:

```
PUBLISH chat:room123 "hello"
```

Subscriber:

```
SUBSCRIBE chat:room123
```

Architecture:

```
User
|
v
Chat Server
|
v
```

```
Redis Pub/Sub
|
v
Connected Clients
```

Used by:

- Discord
- Slack
- messaging apps

---

## 6. Session Storage

Problem:

Store user session state.

Example:

```
login session
authentication tokens
```

Redis solution:

```
SET session:abc123 user_data EX 3600
```

Benefits:

- automatic expiration
- fast lookup
- scalable

Used by:

- web applications
- authentication services

---

## 7. Caching Layer

Problem:

Database becomes overloaded.

Solution:

Use Redis cache.

Architecture:

```
Client
|
```

```
 v
Redis Cache
 |
 v
Database
```

Flow:

1. Check Redis
2. If cache miss → query DB
3. Store in Redis

This reduces DB load dramatically.

---

## 8. Distributed Counters

Problem:

Track metrics in real time.

Examples:

```
page views
likes
downloads
```

Redis solution:

```
INCR page_views
```

Example:

```
pageviews:2026-03-17
```

Used in:

- analytics systems
  - monitoring dashboards
- 

## 9. Event Streaming Systems

Problem:

Process events in real time.

Example events:

```
user_login
purchase
click_event
```

Redis solution:

```
Redis Streams
```

Example:

```
XADD events * user_id 123 action login
```

Workers consume events:

```
XREADGROUP GROUP workers worker1 STREAMS events >
```

This enables **event-driven architectures**.

---

## 10. Real-Time Recommendation Systems

Problem:

Generate personalized recommendations quickly.

Example:

```
recommended products
recommended videos
recommended posts
```

Redis solution:

Use sorted sets and feature storage.

Example:

```
ZADD recommendations:user123 0.98 productA
ZADD recommendations:user123 0.76 productB
```

Get top recommendations:

```
ZREVRANGE recommendations:user123 0 10
```

---

## 11. Distributed Task Scheduler

Problem:

Schedule tasks across multiple machines.

Example:

```
send reminders
run background jobs
refresh caches
```

Redis solution:

```
Sorted Sets with timestamps
```

Example:

```
ZADD tasks timestamp task_id
```

Worker fetches tasks:

```
ZRANGEBYSCORE tasks 0 current_time
```

---

## 12. Geo-Spatial Systems

Problem:

Find nearby objects.

Example:

```
drivers near passenger
restaurants near location
```

Redis solution:

```
GEOADD locations longitude latitude id
```

Query nearby:

```
GEOADIUS locations lon lat 5 km
```

Used by:

- Uber
  - food delivery apps
  - logistics systems
-

# Redis System Design Superpower

Redis provides **building blocks for distributed systems**:

| Problem         | Redis Feature |
|-----------------|---------------|
| Caching         | Strings       |
| Queues          | Lists         |
| Ranking         | Sorted Sets   |
| Messaging       | Pub/Sub       |
| Event Streaming | Streams       |
| Rate Limiting   | Counters      |
| Geo Queries     | Geo Index     |
| Scheduling      | Sorted Sets   |

## The Redis Mental Model

Think of Redis as:

A real-time distributed data structure server

Instead of designing systems like:

App → Database

Modern architecture:

```
App
|
+---- Redis (real-time layer)
|
+---- Database (persistent layer)
```

Redis handles **speed**, database handles **durability**.

## Final Insight

Most large-scale systems follow this pattern:

```
Users
|
```

```

 v
API Servers
 |
+---- Redis Layer
 |
+---- Primary Database
```

Redis acts as:

the performance accelerator for modern systems